

Dictionaries :-

- Dictionaries are used to store data values in key: value pairs.
- ordered, changeable & do not allow duplicates.
- Dictionaries are written with curly brackets & have keys and values.

```
e.g. thisdict = {  
    "brand" : "Ford",  
    "model" : "Mustang",  
    "year" : 1964  
}  
print(thisdict)
```

Output

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

- Dictionary items are presented in key: value pairs and can be referred to by using the key name:-

```
print(thisdict["brand"])
```

» Ford

Duplicates are not allowed

```
dict = {  
    "brand" : "Ford",  
    "brand" : "BMW",  
    "year" : 1964  
}
```

- ∴ Duplicate values will overwrite existing values.
- ∴ Dictionaries cannot have two items with same key.

The values inside dictionary can be of any data type:

eg:-

```
thisdict = {  
    "brand": "Ford",  
    "electric": False,  
    "year": 1964,  
    "colors": ["red", "white", "blue"]  
}
```

Accessing items:

→ print (thisdict ["brand"])
 >> Ford

→ using .get()

→ print (thisdict.get ("year"))
 >> 1964

If the key does not exist: output will be None, there will be no error for that

→ print (thisdict.get ("model"))
 >> None

For retrieving keys and values separately:

→ print (thisdict.keys())

 >> dict_keys (['brand', 'electric', 'year', 'color'])

→ print (thisdict.values())

 >> dict_values (['Ford', False, 1964, ['red', 'white', 'blue']])

get items :-

The items() method will return each item in a dictionary, as tuples in a list.

→ print(thisdict.items())

>> dict_items([('brand', 'Ford'), ('electric', False), ('year', 1964), ('color', ['red', 'blue', 'black'])])

check if key exists :-

if "brand" in thisdict:

print("yes, 'brand' is one of the key")

else:

print("Key does not exist")

Change Values :-

change the value of a specific item by referring to its key name.

thisdict = {

 "brand": "BMW",

 "model": "M5",

 "year": 2025

}

this will change to M4

thisdict["model"] = "M4"

print(thisdict)

Update Dictionary :-

update() method will update the dictionary with items from given arguments.

```
→ thisdict.update({"company": "Audi"})  
                    brand  
print(thisdict)
```

Output

```
{'brand': 'BMW', 'model': 'M5', 'year': 2025}
```

this will get updated

Add Dictionary Items :-

```
→ thisdict["color"] = "red"  
print(thisdict)
```

Output

```
{'brand': 'Audi', 'model': 'M5', 'year': 2025, 'color': 'red'}
```

```
→ thisdict.update({"engine": "V16"})  
print(thisdict)
```

Output

```
{'brand': 'Audi', 'model': 'M4', 'year': 2025, 'color': 'red',  
 'engine': 'V16'}
```

Removing Items :-

1) .pop() method:

removes the item with specified key name.

```
thisdict.pop("year")
```

↳ this will remove year key with its value from dict.

2) .popitem() :-

.popitem() method removes the last inserted item.

thisdict.popitem()

↳ this will remove engine from thisdict.

3) del keyword:-

del keyword removes the item with specified key name & can also delete the dictionary completely.

→ del thisdict["model"]

↳ deletes model key with its value.

→ del thisdict
print(thisdict)

} this will cause error because the dictionary is already deleted.

4) .clear() method:

.clear() method empties the dictionary.

→ thisdict.clear()
print(thisdict)

Loop through dictionary:-

Print all key names in the dictionary

for x in thisdict:
print(x)

OR

for x in thisdict.keys():
print(x)

For printing values:-

for x in thisdict.values():
print(x)

Loop through both keys & values, by using items() method

```
for x, y in thisdict.items():  
    print(x, y) OR print(x, ":", y)
```

Output will be

```
>> brand    BMW  
    model    M5  
    year    2024
```

Copy a dictionary:-

```
my-dict = thisdict.copy()  
print(my-dict)
```

Nested Dictionary:

Dictionary ^{can} contain other dictionaries.

Eg:-

```
myfamily = {  
    "child1": {  
        "name": "Emily",  
        "age": 26  
    },  
    "child2": {  
        "name": "Pondd",  
        "age": 17  
    },  
    "child3": {  
        "name": "Tom",  
        "age": 10  
    }  
}
```

Accessing items in
Nested Dictionary:

```
print(myfamily["child1"]  
      ["name"])
```

>> Emily.

```
person = {
```

```
    "name": "Alok",
```

```
    "age": 21,
```

```
    "skills": ["Python", "AWS", "Devops"]
```

```
}
```

```
print(person["skills"][1])
```

```
>> AWD
```

Sets:-

→ A set is a collection which is unordered, ~~mutable~~, and unordered

→ Set items are unchangeable but you can remove or add new items.

→ Sets are written with curly brackets: {}

→ do not allow duplicate values.

eg:-

```
thisset = {"apple", "banana", "orange"}
```

```
print(thisset)
```

∴ Sets are unordered, so we cannot be sure in which order the item will appear.

NOTE: The values True & 1 are considered the same value in sets & are treated as duplicates.

```
set = {"apple", "Alok", True, 1, 2}
```

↳ these are duplicate values.

Same for False & 0 (both are considered as duplicate values)

A set can contain diff. data Types.

```
set = { "Alok", True, 40 }
```

Accessing Set Items:-

→ We cannot access item in a set by referring to an index or a key.

But we can loop through them.

```
for x in set:  
    print(x)
```

Output

```
Alok  
True  
40
```

Check if item exists:

```
if "True" in set:  
    print("Yes, 'True' is present")
```

```
else:  
    print("Not present")
```

OR

```
print("True" in set)  
print("Banana" not in set)
```

Adding value in set:-

→ `set.add("Mango")`
`print(set)` # This will add mango in the set.

To add items from another set into the current set, use the update method.

eg.

```
cars = {"BMW", "Mercedes", "Audi"}
```

```
set.update(cars)
```

```
print(set) # this will add items from another
```

```
set.update(["mango", "apple"]) set.
```

∴ using .update() method can be used to add any iterable object (tuple, list, dictionary etc)

eg.

```
games = ["Bomberman", "Elden Ring", "ROR"] # list
```

```
set.update(games) # this will add list to a set
```

```
print(set)
```

Remove Set items :

→ To remove an item in a set, use the remove() or discard() method.

e.g.-

```
set.remove("Alok")
```

```
print(set).
```

→ if the item to remove does not exist, this will raise an error.

```
set.discard("banana")
```

```
print(set).
```

→ this will not raise any error if the item tends to be removed does not exist in set.

Two methods for removing items :-

→ .remove()

→ .discard()

We can also use `.pop()` method to remove an item, but this method will remove a random item so we are not sure which item will get removed.

→ the return value of `.pop` method is the removed item.

→ `.clear()` method empties the sets.

```
set.clear()
print(set)
```

» `set()`

→ del keyword will delete the set completely.

```
del set
print(set) # an error will occur.
```

Loop through sets

eg-
fruits = {"apple", "cherry", "orange"}
for x in fruits:
 print(x)

Note: we can not add multiple values or items in a set using `.add()` method.

→ If we want to add multiple values or items we have to use `.update()` method.

Join sets:-

There are several ways to join two or more sets.

→ Union:

returns a new set with all the items from both sets.

eg:-

set1 = {1, 2, 3}

set2 = {"a", "b", "c"}

set3 = set1.union(set2)

print(set3)

>> {1, 'a', 'b', 3, 'c', 2}

Instead of writing `set1.union(set2)` we can use | operator, you will get same result.

set3 = set1 | set2

Join multiple sets:-

myset = set1.union(set2, set3, set4)
OR

myset = set1 | set2 | set3 | set4
print(myset)

Join a set and a tuple :

union method allows you to join set with other data types like lists or tuples.

Eg:-

```
x = {1, 2, 3}
```

```
y = {"Akshat", "Abhay", "Akshat"}
```

```
z = x.union(y)
```

```
print(z)
```

* | operator only allows you to join set with sets.

2) Update

update() method inserts the items in set2 into set1

Eg:

```
x = {"a", "b", "c"}
```

```
y = {1, 2, 3}
```

```
x.update(y)
```

```
print(x)
```

```
>> {1, 2, 'a', 3, 'b', 'c'}
```

∴

Both union() and update() will include only duplicate items.

3) Intersection

→ this method returns a new set, that contains items that are present in both sets.

```
set1 = {"amazon", "apple", "google"}
```

```
set2 = {"audi", "BMW", "apple"}
```

```
set3 = set1.intersection(set2)
```

```
print(set3)
```

```
>> {'apple'}
```

OR

→ we can use & operator instead of intersection() method.

```
set3 = set1 & set2
```

```
print(set3) # output will be same.
```

∴ & operator only works with sets to sets not with other data-types.

4) intersection_update() method :-

→ also keeps only the duplicates, but it will change the original set instead of returning a new set.

```
→ set1.intersection_update(set2)  
print(set1)
```

5) Difference

→ returns a new set that will contain only the items from the set that are not present in other set.

→ Keeps all the item from set1 that are not in set2.


```
set1 = {"apple", "banana", "cherry"}  
set2 = {"google", "microsoft", "apple"}  
set3 = set1.difference(set2)  
print(set3)
```

Here apple is present in both sets, so in set1 apple will be removed & rest items are printed.

```
>> {"banana", "cherry"}
```

```
→ set3 = set2.difference(set1)
```

```
>> {'google', 'microsoft'}
```

```
→ set3 = set1.difference(set1)
```

```
>> set()
```

OR

we can use - operator instead of difference() method

```
→ set3 = set1 - set2
```

```
print(set3) # result will be same.
```

OR

use difference-update() method.

```
→ set1.difference_update(set2)
```

```
print(set1)
```

5) Symmetric-difference() method :

→ this method keeps only the elements that are not present in both sets.

e.g. set1 = {"apple", "banana", "cherry"}
set2 = {"microsoft", "google", "apple"}
set3 = set1.symmetric-difference(set2)
print(set3)

>> {'banana', 'cherry', 'microsoft', 'google'}

OR

→ we can use $\wedge$ operator.

→ set3 = set1 $\wedge$ set2
print(set3)

OR

symmetric-difference-update()

→ set1.symmetric-difference-update(set2)
print(set1)

>> {'banana', 'google', 'cherry', 'microsoft'}

NOTE imp

- update , • union , • intersection , • difference ,
- symmetric-difference along with • intersection-update ,
- difference-update & • symmetric-difference-update

work with iterable objects like lists, tuples and dictionaries. (only keys are considered)

But with operator like | , & , - , ^ work only with sets to sets.

→ method → works with any iterable object
→ operators → work only with sets

Python frozenset:

- frozenset is immutable version of set.
- Like sets, it contains unique, unordered, unchangeable elements.
- Unlike sets, elements cannot be added or removed from a frozenset.

creating a frozenset

- use `frozenset()` constructor to create a frozenset from any iterable.

eg:-

```
x = frozenset ( { "apple", "banana", "cherry" } )  
print(x)  
print(type(x))
```

Here we can write any iterable object.

Output

```
>> frozenset ( { 'cherry', 'apple', 'banana' } )  
<class 'frozenset'>
```

File operations :-

- In python, file operations means reading, writing and managing files (like .txt, .csv etc) using python built in functions.
- Python makes it very easy to handle files using the built in `open()` function.

open() function :

- `open()` function takes two parameters : filename & mode

Four diff. methods (modes) for opening a file:-

`r` = Read - Default value, opens a file for reading, error if the file does not exist.

`w` = Write - opens a file for writing, creates the file if it does not exist.

`a` = Append - opens a file for ~~writing~~ appending, creates the file if not exist.

`x` = Create - creates the specified file, returns an error if the file exists.

`t` = Text - Default value, Text mode.

e.g. `f = open("demoFile.txt")`

To open a file, use the built in `open()` function
→ `open()` returns file object, which has a `read()` method for reading the content of the file.

```
f = open("demofile.txt")  
print(f.read())
```

>> Hello there!

→ If file is in diff. location :-

```
f = open("D:\\myfiles\\hello.txt")  
print(f.read())
```

Using with statement :-

```
with open("hello.txt") as f:  
    print(f.read())
```

∴ It's a good practice to always close the file, if you are not using with statement

For closing:-

```
f.close()
```

=
with open("hello.txt") as f:
 print(f.read(5)) # this will return first 5 char.
of file.

Readlines

You can return one line by using `readline()` method.

```
with open("hello.txt") as f:  
    print(f.readline())
```

>> Hello All....

→ By calling `readline()` two times, you can read the two first line.

By looping through the file, you can read the whole file line by line.

```
→ with open("hello.txt") as f:  
    for x in f:  
        print(x)
```

readlines() method

```
with open("hello.txt") as f:  
    print(f.readlines())
```

→ `['Hello!...\n', 'this is file \n', 'good luck']`

→ `readlines()` method read all lines from a file & return the as list of strings.

Writing in file:-

For writing in a file add `append`, or `write` parameter to the `open()` function.

e.g:-

```
with open("hello.txt", "a") as f:  
    f.write("\nNew line added")
```

~~XXXXX~~

• # adds a new line at the end in a file

e.g. with open("hello.txt", "a") as f:
f.write("\n New line using append")
open and read after the appending
with open("hello.txt") as f:
print(f.read())

Overwrite the existing content :-

→ To overwrite the existing content to the file use w parameter.

with open("hello.txt", "w") as f:
f.write("Prev. content deleted, new added")
open and read the file after the overwriting:
with open("hello.txt") as f:
print(f.read())

Create a new file :-

with open("newfile.txt", "x") as f:
pass
a new empty file is created
(If file already exist, an error will be raised)

with open("new-file.txt", "x") as f:
pass

→ This will create an empty file.
& If you want to add content, instead of pass, write
f.write("Hello!, This is a new file")
with open("new-file.txt") as f:
print(f.read())

Delete a file :-

To delete a file, import OS module and run it's `os.remove()` function

Remove new-file.txt

```
import os  
os.remove("new-file.txt")
```

Check if file exist :-

```
import os  
if os.path.exists("file.txt"):  
    os.remove("file.txt")  
else:  
    print("The file does not exist")
```

Deleting a folder :-

→ To remove entire folder, use `os.rmdir()` method

```
import os  
os.rmdir("my folder")
```

You can only remove empty folder.

BOTO-3

- Boto3 is the official AWS SDK (Software Development Kit) for python.
- It allows your python programs to interact directly with AWS services - like EC2, S3, Lambda, DynamoDB.

Boto3 = Python + AWS = Automation

Interfaces in Boto3

1) Resource :-

- High level object-oriented API.
- Simplifies operations by treating AWS services as python object.

Eg:-

`s3 = boto3.resource('s3')`

(Not available for all AWS services (some don't have resource interfaces))

2) Client :-

- Direct mapping to AWS API calls. more control

Eg:- `s3_client = boto3.client('s3')`

∴ You need to handle JSON responses manually.

AWS CLI
BOTO3

} Scripting

CFT

Terraform

} Templating

↳ Boto3 can be used to write serverless program for lambda & other serverless services

But AWS CLI do not have option for serverless services in AWS.

Creating bucket in S3 using Boto3:

```
import boto3
```

```
S3 = boto3.client('S3', region_name='ap-south-1')
```

```
# This creates S3 client object, which allow you to call  
S3 API directly or other AWS services
```

```
S3.create_bucket(
```

```
    Bucket = 'alok-demo-boto3-bucket',
```

```
    CreateBucketConfiguration = {'LocationConstraint': 'ap-south-1'})
```

```
)
```

```
print("Bucket created successfully")
```

S3.create_bucket() → boto3 method to creates new S3 bucket.

```
CreateBucketConfiguration = {'LocationConstraint': 'ap-south-1'}
```

↳ This specifies the region where you want to create the bucket.

imp * → Its required for any region other than us-east-1

→ If you are creating bucket in us-east-1,
you must omit this parameter.
↓
(to leave)

∴ When one service (AWS) tries to talk to other AWS service, that is possible by using IAM role which needs to be created first.